

1. Effizienz von Algorithmen

1.1. Asymptotische Notation

$g = O(f)$	g wächst maximal so schnell wie f $0 \leq \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} < \infty$ $O(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \exists c > 0, n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \leq cf(n)\}$
$g = \Omega(f)$	g wächst mindestens so schnell wie f $0 < \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} \leq \infty$ $\Omega(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \exists c > 0, n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \geq cf(n)\}$
$g = \Theta(f)$	g wächst genauso schnell wie f $0 < \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} < \infty$ $\Theta(f) = O(f) \cap \Omega(f)$
$g = o(f)$	g wächst langsamer als f $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$ $o(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \forall c > 0 \exists n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \leq cf(n)\}$
$g = \omega(f)$	g wächst schneller als f $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \infty$ $\omega(f) := \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \forall c > 0 \exists n_0 \in \mathbb{N} : \forall n > n_0 : g(n) \geq cf(n)\}$

1.1.1 Rechenregeln

Für O, Ω, Θ :

- $cf(n) = O(f(n))$
- $O(f(n)) + O(g(n)) = O(f(n) + g(n)) = O(\max(f(n), g(n)))$
- $O(f(n))O(g(n)) = O(f(n)g(n))$
- $\log n < \sqrt{n} < n < n \log n < n^{3/2} < n^2 < n^3 < 1.1^n < 2^n < n!$

2. Wichtige Algorithmen

2.1. Sortieralgorithmen

2.1.1 InsertionSort

- Wähle $\text{num} = \text{input}[i]$
- Vergleiche alle Indices $j < i$ und schiebe num nach $\text{input}[j]$, wenn $\text{input}[j] < \text{num}$

Laufzeit	Best case: $O(n)$ falls Input sortiert
	Worst case: $O(n^2)$ falls Input absteigend sortiert
	$O(hn)$ falls ein Element höchstens h Elemente von seiner Zielposition entfernt ist

2.2. Suchalgorithmen

2.2.1 Binary Search

Annahme: Liste sortiert, gesucht x .

- Vergleiche x mit dem Wert, der in der Mitte steht
 - Bei Gleichheit sind wir fertig
 - Falls x kleiner als der mittlere Wert, gehe zu
 - mit dem linken Teil des Arrays

- Falls x größer als der mittlere Wert, gehe zu
 - mit dem rechten Teil des Arrays

Laufzeit	Best case: $O(1)$ falls mittlere Wert gleich gesuchter Wert
	Worst case: $O(\log_2 n)$